

COP3330C Module 15 Programming Assignment 7

Objective: In this assignment, we will modify our MVC-based book manager to use an SQL database. This transition will demonstrate the versatility of the MVC pattern by showing how its model implementation can be easily switched from an in-memory representation to a persistent data store.

Design Notes:

The project introduces two new classes: a **DatabaseUtility** class and a **BookRepository** class.

- The DatabaseUtility class encapsulates the database connection logic and provides a static method for obtaining a database connection. This class modularizes the database-related concerns, adhering to the principles of separation of concerns and single responsibility. The BookRepository class acts as an intermediary between the BookController and the database, implementing repository-pattern logic to handle CRUD operations. The BookRepository centralizes data access methods, further decoupling database logic from the controller and improving maintainability.

By transitioning to a database, serialization is no longer necessary because the database provides robust, efficient, and scalable data persistence mechanisms. This approach aligns with modern application design practices.

The BookController class has been updated to delegate the CRUD operations to the BookRepository, which manages the interaction with the SQL database.

BookView Class Deprecation: With the transition to an SQL database, the BookView class is no longer used or necessary. The current implementation does not require a separate view component for presenting data, as the focus is on demonstrating the persistence and scalability of the model through database integration. Any necessary output for testing can be done directly via console logging or other basic means. A more useful view could be easily added by implementing REST APIs to facilitate multiple types of user interfaces.

These design changes demonstrate how the MVC framework supports scalability and robustness by isolating data management changes within the Model and leveraging helper classes like DatabaseUtility and BookRepository. This reinforces the adaptability of the design to accommodate growth and evolving requirements and exemplifies how decoupling data persistence from the rest of the application facilitates easy adaptation, making it possible to change the data source without modifying the application logic.

Implementation Notes:

This project uses the H2 database (<https://www.h2database.com>), an open-source solution that provides a straightforward in-memory SQL database. Adding H2 to the project requires only a single dependency in the Maven build configuration file, pom.xml, often referred to as "the POM file."

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.1.214</version>
</dependency>
```

This dependency instructs Maven to download and include the necessary libraries, which can then be utilized via connection statements in the Java source code. While H2 operates as an in-memory database in this setup, it is designed for easy transition to other databases, such as PostgreSQL or Microsoft SQL Server, without requiring changes to the existing SQL code.

H2 offers a simple and efficient solution for testing code, reducing the need for complex configurations and external software dependencies.

Submission:

The GitHub Classroom repository provides starter code that implements most of the book management application.

NOTE: In this version of the project, book additions are now hardcoded in the main method of the application instead of using user input from the console. Add any necessary book entries to your code to test various scenarios, such as different authors, book titles, and edge cases like empty strings or duplicate book titles for thorough testing of the BookRepository functionality.

Your task is to complete the TODO: items in the BookRepository class to implement the database operations.

- **save(Book book) Method:**
Implement a prepared statement to insert a new book into the books table.
Use `connection.prepareStatement()` with an INSERT SQL query.
Set the title and author fields using `preparedStatement.setString()`.
Retrieve the generated key to set the book's ID.
- **findAll() Method:**
Implement a method to retrieve all books from the database.
Use `connection.createStatement()` with a SELECT SQL query to fetch all records from the books table.

Loop through the ResultSet to create Book objects and add them to a list.
Return the populated list.

- **findById(Long id) Method:**
Implement a method to find a specific book by its ID.
Use `connection.prepareStatement()` with a SELECT query and a WHERE clause to find a book by the given ID.
Extract the book details from the ResultSet and return the Book object.
- **delete(Book book) Method:**
Implement a method to delete a specific book using its ID.
Use `connection.prepareStatement()` with a DELETE query.
Set the book's ID in the prepared statement to delete the corresponding record from the books table.

Ensure that all SQL statements use prepared statements to prevent SQL injection attacks and that each method includes appropriate exception handling (try-catch blocks) to handle potential SQLException errors gracefully. Run the application to verify that the CRUD (Create, Read, Update, Delete) functionality is working as expected. Submit your solution to the repo (retain the original project structure) and a screen snip showing your program execution.